

ControlPlane

Business Proposal

Enterprises are putting AI in front of customers, staff and money. The cost of checking what it says is becoming a line item nobody planned for, and most of that line item is people. This is a proposal to spend it deliberately.

81 to 98%

of the assurance bill is human review time, not compute

37.5x

what one human review costs against the dearest automated check

1.59x

more expected loss handled from the same reviewer hours

8 weeks

to a decision, on one route, with no enforcement risk

Team

BurningShadows

Prepared for

Enterprise AI platform and risk owners

Ask

One route. One shadow pilot. Eight weeks

Executive summary

Every enterprise deploying AI assistants has quietly acquired a new operating cost. Somebody has to check what the model says before it reaches a customer or moves money. That cost is growing faster than the AI spend it protects, and almost nobody is managing it.

The two things organisations do today both waste money. Running a strong checker on every answer is thorough and can cost more than generating the answer did. Sampling one answer in ten is cheap and blind, because it spends the same effort on a question about opening hours as on an instruction to transfer funds.

ControlPlane prices the difference. It estimates what it would cost the business if a given answer were wrong, and buys checking only where the loss prevented is worth more than the check. A safety floor sits underneath that the budget cannot switch off. Everything lands in a tamper evident record that an auditor can recompute.

92.8% of the protection of checking every answer, for 10% of the compute	4.29x more loss left on the table by doing nothing at all	2 of 15 high value cases dropped by our review order against 15 by first in first out	0% of verdicts change when every price assumption is swung four fold
--	---	---	--

The finding that shapes this proposal

We set out to save compute. The measurement said something more useful. **Between 81 and 98 paise in every assurance rupee is human review time, not machine time.** One completed review costs ₹120 against ₹3.20 for the most expensive automated check. Every vendor in this category competes on the 2 to 19 paise and nobody is managing the rest.

That changes what is worth buying. The compute saving is real and it is the smaller prize. The larger prize is deciding which cases reach a person at all, and in what order, because review capacity is fixed and the queue is already oversubscribed.

What we are asking for. One customer facing route, in shadow mode, for eight weeks. No enforcement, no latency risk, no change to the existing model. At the end you will have your own numbers for what assurance costs you today and what it would cost run this way.

Problem framing

The problem is not that AI makes mistakes. It is that checking for mistakes has no budget, no owner and no way to prove it worked.

The cost nobody planned for

An assistant answering customer questions produces thousands of answers a day. A fraction are wrong in ways that matter. Finding them costs money in two places, and the second is much larger than the first.

What it costs to check one answer	Price	Who pays it
Rules and pattern scan	₹0.02	Compute budget
Small model checks	₹0.18	Compute budget
Language model judge	₹3.20	Compute budget
One completed human review	₹120.00	Headcount budget

Prices from `config/economics.yaml` . The reviewer rate is a ₹10,00,000 base at a 1.8 times loading over 1,500 productive hours, which sits at the top of a defensible band.

A review is **37.5 times the most expensive automated check**. It follows that once checking escalates even a small share of traffic to people, the people dominate the bill. Our measurement confirms it across every budget setting we tested.

Three failures that follow

OVERSPENDING

Checking everything

Adds 900 milliseconds to every answer and, at two million interactions a year, costs ₹64 lakh in compute before a single reviewer is paid.

UNDERSPENDING

Sampling blindly

A fixed sample cannot tell a question about opening hours from an instruction to move money, so it protects the cheap and misses the expensive.

UNPROVABLE

No defensible record

When a regulator asks why one answer was released, a log line is not an answer. The arithmetic behind the decision is what is needed, and it is usually gone.

The queue is already failing quietly

In our evaluation the review queue is **1.5 times oversubscribed** at the tightest budget and up to 3.4 times at full coverage. Keeping up with arrivals needs 3.0 reviewers against the two staffed. Roughly a third of raised cases are shed unreviewed. Most organisations are already shedding like this. They simply do not measure which cases they lose.

The question is not whether cases get dropped. They already are. The question is whether the ones dropped are the cheap ones or the expensive ones.

Business opportunity

The budget for this already exists inside most enterprises. It is sitting in manual review headcount and in unmeasured incident cost, not in a tooling line.

Why now

Three things arrive together. AI assistants are moving from pilots into customer facing and money touching workflows. Regulation is starting to ask for evidence of control rather than assertion of it. And the first wave of guardrail tooling has made checking possible without making it affordable, which is exactly the gap a budgeting layer fills.

Where the value sits

Value	What it is worth	Confidence today
Reviewer hours redirected	The same review capacity handles 1.59 times more expected loss, and drops 2 high value cases instead of 15	Measured on our corpus
Compute avoided	92.8% of the protection of checking everything, for 10% of the compute	Measured on our corpus
Incidents avoided	Doing nothing leaves 4.29 times more residual loss than allocating does on the same traffic	Depends on the customer's own consequence prices
Audit cost avoided	Every decision recomputable from its own record, with the policy version attached	Built and verified

What we are not claiming. The rupee value of loss averted is a property of the consequence prices we assumed, not evidence of a customer saving. Those prices are configuration and they are the first thing a pilot replaces with the customer's own numbers. What we defend today is the cost structure, the measured catch rates and the guarantee.

Proposed solution

A gateway that sits between your application and whatever model answers it, and decides how much checking each answer is worth.

What it does, in the order it does it

Step	What happens
Refuse early	A cheap scan reads the prompt before the model runs. An obvious attack is refused there and costs neither generation nor checking.
Score the answer	Cheap checks score the answer on five kinds of harm for a combined 20 paise, and those scores are converted into probabilities.
Apply the floor	Each route has a safety threshold above which a check is compulsory. The budget cannot switch this off.
Price the choice	Below the floor, the system buys the expensive check only where the loss it would prevent beats what it costs.
Split words from actions	Text can be shown while checking runs. Actions that move money or change records wait behind their own gate.
Prioritise people	Whatever needs a person is queued by value per reviewer minute against the deadline, not first come first served.
Record everything	Every decision, its inputs and its price go into a chained record where a later edit is detectable.

What integration actually involves

The gateway presents the same interface your application already calls, so integration is a change of base URL rather than a rewrite. It reads only the request, the answer, any supplied context and any proposed actions. It needs no access to model internals, which means it works in front of a model you rent as well as one you host, and it can run entirely inside your own network where data residency requires that.

The commercial consequence of that design. You are not locked to a model vendor, a detector vendor or a cloud. The system needs three numbers about any checker you plug in, which are its cost, its latency and how much it catches. Change the checker and the economics recalculate themselves.

Target users and buyers

Four stakeholders have to agree, and they care about different things. Each has a different question and a different piece of evidence that answers it.

Stakeholder	Decision they own	What this answers for them
Economic buyer CFO or FinOps lead	The assurance budget and reviewer headcount	Total cost of assurance split into compute and attention, with cost per thousand interactions reported as a first class metric. The budget is set by them and held by the system.
Risk owner Chief Risk or Compliance	The minimum control and the audit evidence	A per route safety floor with a stated bound, jurisdiction specific policy packs, and every override in a verifiable chain. Crucially, price does not enter the release rule.
Technical owner Head of AI Platform	Integration, portability and reliability	A drop in gateway on the interface you already use, detector adapters that are one method, admission control under load, and a load tested failure mode.
Operational user Head of Support or Review	Reviewer workload and what gets dropped	A queue ordered by value against deadline rather than arrival, with shed rate and deadline misses reported rather than hidden.

The economic buyer is usually the Head of AI Platform or the Chief Risk Officer, and the budget usually already exists as manual review headcount rather than as a tooling line. That matters for how this is sold. It is a reallocation of an existing cost, not a new one.

Where it fits first

The strongest early fit is a regulated enterprise running a customer facing assistant alongside an assistant that can take an action. Banking, insurance, telecom and large scale consumer support all have the same shape. High volume, a real consequence table, an existing review desk, and a regulator who will eventually ask for evidence.

Customer use case

Two requests arrive one minute apart at the same support assistant. A fixed rule treats them the same. Here is what happens instead.

REQUEST ONE

A routine ticket lookup

A customer asks for the status of an open ticket. The cheap checks score the answer well below the safety threshold. The expensive judge would prevent very little extra loss and would cost far more than that, so it is not bought. The answer is released.

Cost of assurance. 20 paise. Delay added. None the customer notices.

REQUEST TWO

A payment instruction

A request on the finance route carries a transfer, and the model echoes back a secret its source said must never be repeated. Risk lands above the safety threshold, so a check is compulsory before any price is considered. The judge is bought, the answer is blocked, the transfer is denied, and the case is queued for a person.

Cost of assurance. ₹3.20. Loss on the table. ₹69,357.

The point of the pair. Neither request was treated as average. The first was cheap because nothing was at stake. The second was expensive because a great deal was, and the system could afford it precisely because the first one did not spend.

What the customer never sees

In the second case the customer receives a message saying the request has been passed to a colleague. They do not see the model's original answer. The business does not lose the money, and the review desk receives a case tagged with what it is worth, so it is worked before the routine ones that arrived earlier.

Business case

The unit economics are input driven, and every input is exposed in the audit record so finance and risk can challenge it rather than accept it.

Input	Where it comes from	Status today
Reviewer rate and handling time	Invoice and queue study	Defensible. This is a real number a CFO already owns.
Check price	Provider token prices	Defensible once a real provider adapter is in place.
Catch rate	Measured from reviewer labels plus a sampled audit	Measured. The judge caught 0.930 against 0.880 configured over 365 observations.
Consequence of each harm	Finance and risk, as a low, base and high range	Assumption, but bounded. Across a 0.25 to 4 times swing, 10.9% of spending decisions move and no verdict does.

Status is deliberately blunt. Two of these four are the customer's own numbers and one is still an assumption.

What it costs to run

At two million interactions a year, checking every answer with the strongest tier costs ₹64 lakh in compute. The same protection, allocated, costs ₹6.4 lakh. That is the headline saving and it is the smaller half of the case.

₹64 lakh a year in compute to check every answer at two million interactions	₹6.4 lakh a year for 92.8% of the same protection	81 to 98% of the remaining bill is reviewer time, which is the real target
--	---	--

The two baselines a customer already pays

Doing nothing. With no verification, residual loss on the same traffic is 4.29 times what allocation leaves. The customer's equivalent is their own incident rate multiplied by their own consequence range, which a two week study produces.

Blanket manual review. Reviewing every flagged case at current flag rates is what the oversubscription describes. Most organisations are already shedding. They do not measure which cases they lose.

Stated plainly, because it is the strongest objection. At the tightest budget a well tuned blanket cheap check averts 2.4% more than we do for a fraction of the compute spend. Our advantage on compute alone is small, and once both policies are charged for the reviewer minutes they generate it is smaller still. The compute ratio in that baseline's favour falls from 1.82 times to 1.035 times once attention is counted, and that happens because the attention bill dwarfs compute for both, not because allocation improved. **Where we beat the obvious alternative is on the resource that costs the money.**

Expected impact

Where	Effect measured on held out traffic
Compute spend	92.8% of the protection of checking everything, for 10.0% of the cost. At an 80% budget it beats checking everything, averting more loss for 20% less compute.
Reviewer productivity	1.59 times more expected loss handled from the same reviewer hours. High value cases dropped fall from 15 to 2. Deadline misses fall from 139 to 65.
Customer experience	Text is never held behind a verdict, only actions are. Under overload the system refuses cleanly rather than slowing everyone, holding the effect tail at 104 milliseconds against 1,508.
Audit readiness	Every decision recomputable from its own record. In the evaluation run, 1,500 of 1,500 decisions and 224 of 224 proposed actions were logged in one verified chain.
Risk posture	A stated release bound per route, held on held out data, that the budget is not permitted to relax.

Differentiation

We are not competing with detectors. We are the layer that decides which detector to buy, and when.

Approach	What it does	What it leaves unanswered
Guardrail and moderation tools	Classify text as safe or unsafe, usually well	How much to spend on this answer, and what it costs the business if wrong
LLM as judge platforms	Apply a strong model to check output	Which requests are worth that price, and what happens when the budget runs out
Observability and evaluation suites	Report what happened after the fact	Anything at request time. They do not gate an action
Manual review desks	Apply judgement where it matters	Which cases reach them, in what order, and which are being dropped
ControlPlane	Prices the check, enforces a floor, orders the queue and proves the decision	Detection quality itself, which is why any detector plugs in behind one adapter

The defensible position. Detectors will keep improving and getting cheaper, and that helps us rather than threatening us. A better detector reports a higher catch rate and the allocator automatically buys more of it. What does not commoditise is the budgeting, the guarantee and the audit trail around whichever detectors a customer is allowed to run.

Commercial approach

Element	Proposal
Deployment	Self hosted alongside the customer's existing gateway. It presents the interface the application already calls, so integration is a base URL change.
Pricing	Per thousand interactions assessed, with the assurance budget set by the customer. Cost per thousand is already a reported metric, so the buyer sees the meter from day one.
Land	One customer facing route in shadow mode. No enforcement and no latency risk.
Expand	Add the route that can take an action, then enable enforcement on capped effects with human approval.
Value narrative	Reviewer hours redirected, not GPU spend saved. The buyer is whoever owns the review function.

Pilot plan

Eight weeks, one support route and one route that can take an action. Nothing is enforced until week seven, and nothing is enforced at all without approval.

Phase	What happens	What you get
Weeks 1 to 2 Shadow	Run alongside the existing stack. Sample outcomes, label detector catches, estimate your consequence ranges, measure latency. No enforcement.	Your own baseline. What assurance costs you today, split into compute and attention.
Weeks 3 to 4 Calibrate	Fit the calibration maps and select the release threshold on your traffic, using your labels.	A release bound stated on your own data rather than on ours.
Weeks 5 to 6 Compare	Compare a fixed rate policy against allocation offline, at equal spend and at equal reviewer minutes.	An honest answer on whether allocation helps on your traffic. It does not always.
Weeks 7 to 8 Enforce	Enable holds for reversible or financially capped effects, with human approval on every one.	A working control with an audit trail, and a runbook your incident owner has signed.

Promotion to production requires all of these

- Allocation beats the fixed rate baseline at equal spend across agreed budget bands, with intervals rather than point estimates.
- The route release bound stays under the approved threshold on fresh labels.
- Every proposed action has an audit record, and the record sink meets your retention requirements.
- Added latency at the 99th percentile stays inside the route service level objective.
- Decisions stay stable across the approved consequence ranges.
- The incident and override owners have approved the runbook.

Success metrics

Six numbers decide whether this worked, and all six are already reported by the system rather than assembled afterwards.

Metric	What it tells you	Target for the pilot
Cost per thousand interactions	Total assurance cost, split into compute and attention	Below your current blended cost
Escaped harm rate	Share of released answers that carried harm	Under the approved bound on fresh labels
Expected loss served per reviewer hour	Whether review capacity is going to the right cases	Above the first in first out baseline on your queue
High value cases shed	Which cases you are losing when capacity runs out	Materially below today's rate
Added latency at the 99th percentile	Whether the control is visible to customers	Inside the route service level objective
Audit completeness	Share of decisions and actions with a verifiable record	100%

Phased roadmap

Phase	Scope	Evidence that closes the phase
Now Prototype	Offline gateway, allocation, safety floor, review queue, effect gating, admission control, audit chain	Reproducible commands and an honest baseline comparison. Complete.
Phase 1 Shadow pilot	Real detector adapters, a provider adapter, a sampled audit slice on live traffic	Catch rates and calibration fitted on the customer's own route, plus a latency benchmark
Phase 2 Controlled effects	Reviewer identity, capped held actions, durable effect leases with restart recovery	No unaudited actions and an approved override workflow
Phase 3 Scale	Shared calibration artifacts across workers, signed policy releases, external audit sink, drift detection	Multi worker load and recovery tests, and policy deployment controls

Risks and mitigations

Risk	Consequence	Mitigation and stop condition
Consequence prices steer the wrong traffic	False confidence in the allocation	Measured. 10.9% of spending decisions move across a 0.25 to 4 times band against a 20% stop condition, and no verdict moves. The worst single draw reaches 27.7% and does breach it.
Reviewer capacity is the real constraint	The system optimises the smaller budget	Confirmed, and it is. Attention is 81 to 98% of total cost. This is why the queue ordering is part of the product rather than an afterthought.
Selective labels inflate the catch rate	The economics look better than they are	A fixed rate audit of released rows plus a random sampling reserve, both already implemented and costed against queue performance.
Traffic drift invalidates the bound	Missed checks rise silently	Window monitoring, label refresh and wider coverage. Drift detection is Phase 3 and is not built today.
The safety floor exceeds the budget	The budget target is unmeetable	Report infeasibility. The floor wins and the overspend is declared. The bound is never relaxed silently.
Policy configuration goes stale	Wrong behaviour in a jurisdiction	Effective dates, signed releases, dual approval, and the policy hash recorded on every decision.

Current validation boundaries

The prototype is evaluated on synthetic traffic with configured consequence prices, and on five public corpora. That is enough to demonstrate the mechanism and its economics. It is not a customer result, and the following would have to close before production.

- Calibration is fitted on synthetic traffic and does not transfer. Roughly 500 labelled rows per route, about four weeks of one reviewer, would refit it.
- The expensive judge is a deterministic stand in. The adapter exists and two real adapters are written against it, but a production judge must be built and its catch rate re measured.
- Drift is not detected today. The stated bound holds only for the distribution it was calibrated on.
- The effect gate decides and records but does not hold a lock across a crash. Irreversible actions need a durable lease before enforcement.
- The record store assumes a single process. Multiple workers need shared state.

Closing recommendation

Approve an eight week shadow pilot on one customer facing route, with no enforcement.

The reason to start in shadow mode is that it costs almost nothing and it settles the argument with your own data. In two weeks you will know what assurance actually costs you today, split between compute and people, which most organisations cannot currently state. In eight weeks you will know whether allocating that spend beats what you do now on your traffic, and you will have a release bound measured on your own labels rather than on ours.

We will also tell you if the answer is no. Allocation helps under conditions that are computable in advance, and it does not help outside them. On one of the five public corpora we tested, it won on only one budget setting out of seven, and we report that alongside the results where it won. A pilot that concludes the honest answer is a fixed rate policy on your traffic is a successful pilot, and it will have cost you eight weeks and no enforcement risk to find out.

The ask. One route. One shadow pilot. Eight weeks. At the end you own the numbers, the calibration fitted to your traffic, and a decision you can defend to a regulator.